CANDY
DIALOGUE ENGINE

Adding Dialogue Modes

1.1.0

WARNING: Candy_Engine.gd is overwritten during updates. Always document any modifications you make to it, so that you can re-add them later.

Adding dialogue modes can be done easily.

Main Code

First, open the **Candy_Engine.gd** script, navigate to the "Engine" region, and find the **display_line()** function.

```
func display_line(current_conversation, current_block, speech_data: Dictionary, dialogue_text:
        String, line_data, chosen_variant: String, text_direction: String) -> void:
        #@ Unpack spoken data:
        var speaker_ref: String = resolve_value(speech_data.get("Reference", ""))
        var portrait_raw: String = speech_data.get("Portrait", "")
        var play_portrait: String = resolve_value(speech_data.get("PortraitPlay", "1"))
        var disposition: String = resolve_value(speech_data.get("Disposition", ""))
        var voice_raw: String = speech_data.get("Voice", "")
        var force_bubble: String = resolve_value(speech_data.get("ForceBubbles", "0"))
        var force_portrait: String = resolve_value(speech_data.get("ForcePortrait", "0"))
        var tts: int = int(resolve_value(speech_data.get("TTS", "0")))
        var direction = resolve_value(text_direction).to_lower()
```

The **display_line()** function is called by the **process_lines()** function it processes a spoken line. Note that a the process_lines() function does a lot of pre-processing before calling **display_line()**, so it might be worth having a look at it too, to understand how everything is handled.

In any case, in the **display_line()** function, scroll down to the 'match style' statement:

```
#@ Dialogue Mode:

match style:
        "Box":
                var dialogue_box = get_node(ui_elements_paths["dialogue_box_path"])

                dialogue_box.visible = true
```

```
                #@ Speaker label shows DisplayName:
                dialogue_box.speaker_node.text = speaker_display
                dialogue_text = dialogue_text + box_end_of_line_string
```

The 'match style' statement features a match case for every dialogue style, such as "Box", "Bubbles", "Subtitles", etc.

The style used is defined by 'var **dialogue_mode**' in the Settings region of **Candy_Engine.gd**.

Adding a new dialogue mode is as simple as adding a new match case, then writing code for it. To get started, it's best to look at other modes to understand what logic is part of a dialogue mode.

First, you'll want to choose a mode that is similar to yours. For example, "Box" makes use of the dialogue box and, unlike other modes, features logic to display the speaker's name in a separate RichTextLabel from the spoken text, as well as a portrait.

"Subtitles" and "Bubbles", on the other hand, contain code to optionally include the speaker's name in the same string as the spoken text.

Furthermore, "Bubbles" features special logic for displaying text in a speech bubble, attached to the character sprite or mesh, and which expand to fit the text.

Meanwhile "Chat" displays dialogue in a chatbox-style UI, where previous lines remain visible and can be scrolled (as opposed to the other modes, which display only one line at any time).

If the dialogue mode you want to create is similar to any of the existing modes, you're in luck: that will make things considerably easier. But if your dialogue mode idea doesn't match any of them, it isn't much of a problem as long as you have a solid idea what you want to achieve: you'll just need to see what code from other modes can be copied, and what code must be replaced with entirely new logic.

We're not going to go into details: you can look at the code yourself. But here is a list of the various steps that most dialogue modes include, and which you should probably remember to include:

- Where and how the speaker's name is displayed.

- Where and how the spoken text is displayed.

- Text layout alignment based on text direction (align right for languages written right-to-left).

- Special behavior for VN mode, such as highlighting the speaker's bust.

- Portrait handling, if your mode includes portrait display.

- Audio handling: whether or not to use TTS, or whether or not to use voice files.

- Lexicon formatting.

- The logic to use for displaying the text (can probably be copied from one of the other modes).

- A suspension check, if you want to allow pausing while a line is being displayed.

- Stopping the VN speaker bust highlight (if you used it).

- Stopping voice playback.

- Logging the line that was just displayed to the engine instance's own dialogue_history dictionary, and to the shared general_dialogue_history dictionary.

Order is important. For example, Audio handling should come last before the logic that actually writes the line on screen for two reasons: 1) audio should start just as the line begins displaying, and 2) if you use TTS in simple mode, all BBCode formatting should be resolved before then.

Lexicon formatting should also happen after the segment that determines where and how the line is displayed, to avoid conflicts with BBCode tags.

Paths

If your custom dialogue modes use their own UI elements, you'll need to add paths for them in the script.

Simply add a key in the **ui_elements_paths** in **Candy_Engine.gd**.

Commands

If your custom dialogue modes use their own UI elements, you may need to add code to some commands to support them.

You should mainly look at the §Clear and the §Hide commands, in the commands() function.

Media commands may also need additions, as they alter the z_index of some dialogue display elements (by default, they only affect the dialogue box, portrait box, subtitles, and chatbox, but don't affect speech bubbles or bark bubbles).

Note that for some commands, such as §Clear and §Hide, you may want to add additional input data fields to them in the Candy Dialogue Creator, or just re-use the default ones if you don't expect this to pose any kind of conflict.

For example, if you're confident that there will never be a case where you want to clear your custom dialogue mode's UI but not the dialogue box, you could just modify the §Clear command to clear your custom UI if "Box" = 'true'.